Genereren van fractalen met CUDA

Geavanceerde Computerarchitectuur

Victor Deforce en Cían Olieslagers

May 14, 2025

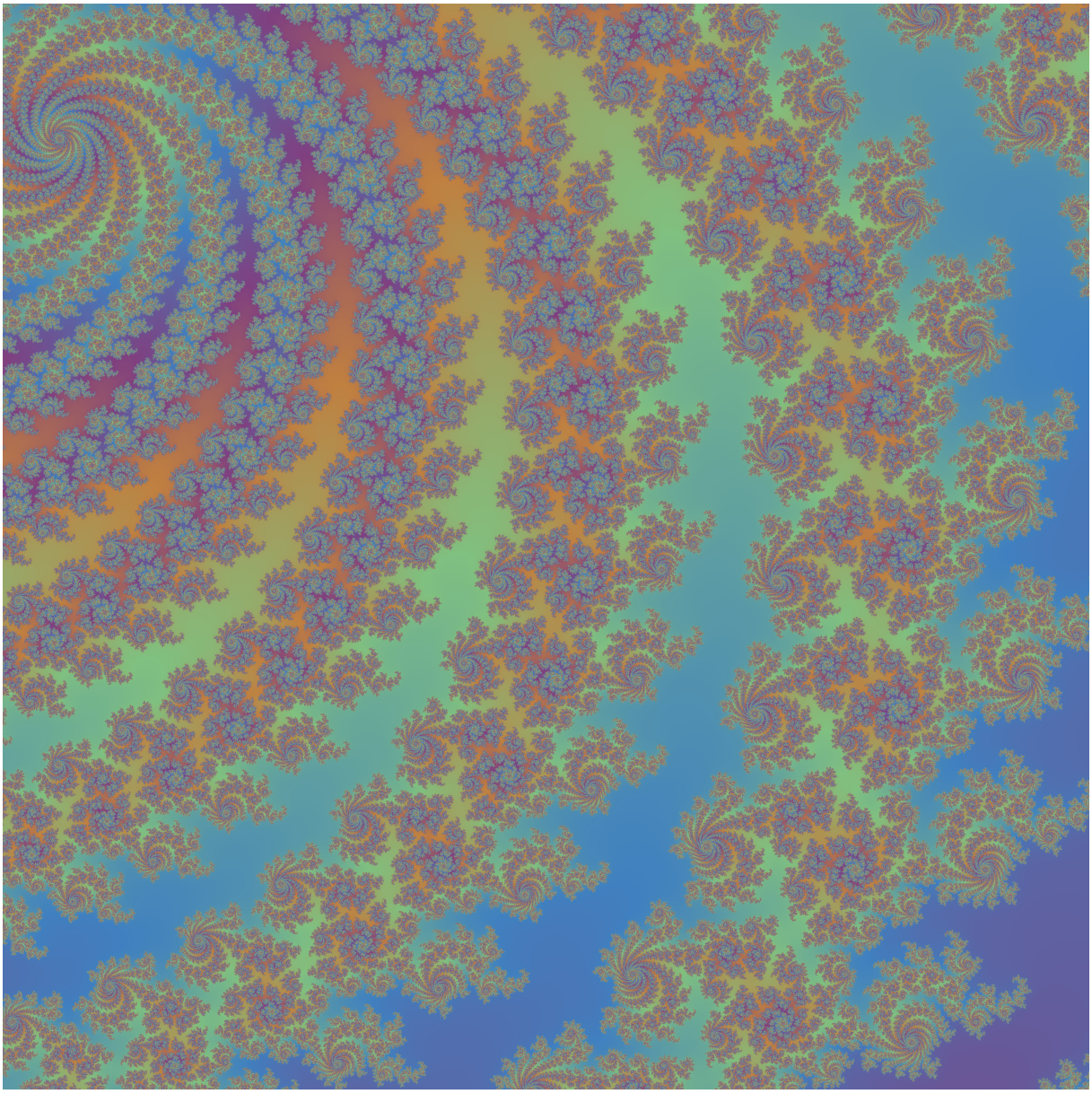

Figure 1: Ingezoomde image Mandelbrot Fractaal

Contents

1 Inleiding **3**

2 Context en achtergrond **3**
2.1 Fractalen . . . 3
2.2 Gebruikte concepten van uit de les . . . 3
2.2.1 Memory types . . . 3
2.2.2 Striding . . . 4
2.2.3 Dynamic parallellism . . . 4

3 Implementatie **4**
3.1 Basis implementatie . . . 4
3.2 Dynamic parallellism uitbreiding . . . 4

4 Resultaten **6**
4.1 Tijdswinst . . . 6
4.2 Gegenereerde fractalen . . . 8
4.2.1 Mandelbrot-verzameling . . . 8
4.2.2 Tricorn . . . 8
4.2.3 Burning Ship . . . 9
4.2.4 Multibrot . . . 10
4.2.5 Julia-verzameling . . . 10

5 Mogelijke verbeteringen **11**
5.1 Implementatie van striding . . . 11
5.2 Memory . . . 11

6 Besluit **11**

7 Appendix - code **12**

1 Inleiding

Het genereren van fractalen is computationeel intensief, vooral bij hoge resoluties. Omdat elke pixel onafhankelijk berekend kan worden, leent dit probleem zich uitstekend tot parallelle verwerking op een GPU. In dit verslag wordt onderzocht hoe fractaal afbeelden met CUDA efficiënt berekent kunnen worden.

2 Context en achtergrond

2.1 Fractalen

Fractalen worden bepaald door per punt in het complex vlak c te berekenen of het tot de set behoort of niet. Iteratief wordt een formule toegepast die start vanuit een initiële waarde z_0 (vaak 0 of de coördinaat zelf).
Bij elke iteratie wordt gecontroleerd of de modulus van z_n groter wordt dan een grenswaarde (meestal 2). Als dit binnen een maximum aantal iteraties gebeurt, behoort het punt niet tot de set en krijgt het een kleur op basis van het aantal iteraties. Anders wordt het zwart gekleurd.
Deze berekeningen kunnen volledig onafhankelijk gebeuren: elke GPU-thread berekent één punt onafhankelijk van de rest.

De fractalen die we implementeerden gebruiken elk een andere iteratieformule:

- **Mandelbrot**: $z_{n+1} = z_n^2 + c$ met $z_0 = 0$
- **Tricorn**: $z_{n+1} = \overline{z_n}^2 + c$
- **Burning Ship**: $z_{n+1} = (|\mathrm{Re}(z_n)| + i|\mathrm{Im}(z_n)|)^2 + c$
- **Multibrot (graad 3)**: $z_{n+1} = z_n^3 + c$
- **Julia**: $z_{n+1} = z_n^2 + c$, met een vaste c en z_0 als pixelcoördinaat

2.2 Gebruikte concepten van uit de les

2.2.1 Memory types

CUDA biedt verschillende geheugentypes, elk met specifieke eigenschappen rond snelheid, bereik en gebruik:

1. Global memory:
   Groot en toegankelijk voor alle threads, maar met hoge toegangstijd. Wordt vaak standaard gebruikt (zoals in onze fractalcode met cudaMalloc).
2. Shared memory:
   Klein en snel geheugen dat gedeeld wordt binnen één block. Ideaal om vaak gebruikte data te cachen en communicatie tussen threads van hetzelfde block te faciliteren.

2.2.2 Striding

Striding is een programmeertechniek waarbij één thread meerdere data-elementen verwerkt door ze met vaste stappen (strides) door te lopen. Dit is vooral nuttig wanneer het aantal beschikbare threads kleiner is dan het aantal te verwerken elementen.
In plaats van dat elke thread slechts één element verwerkt, herhaalt de thread zijn werk meerdere keren op verschillende posities in de data.

```
int index = threadIdx.x + blockIdx.x * blockDim.x;
int stride = blockDim.x * gridDim.x;

for (int i = index; i < size; i += stride) {
    // Verwerk element i
}
```

Listing 1: Voorbeeld van striding in een kernel

2.2.3 Dynamic parallellism

Dynamic parallelism laat toe dat een kernel op de GPU zelf een andere kernel opstart, zonder tussenkomst van de CPU. Dit is handig voor algoritmes met onbekende of ongelijke werkverdeling, zoals recursieve fractalen of adaptieve grids.

3 Implementatie

3.1 Basis implementatie

Voor elke pixel bepalen we de coördinaten in het imaginair vlak. De coördinaten van de pixel worden iteratief door de functie van de gewenste fractaal gevoerd. Als het punt hierdoor richting oneindig gaat, beschouwen we deze geen deel van de set. We kleuren de pixel op basis van het aantal iteraties dat nodig was om een modulus groter dan twee te bereiken. Als echter het punt niet naar oneindig gaat en na N_max iteraties nog steeds een modulus heeft kleiner dan twee, dan behoort het punt tot de set en kleuren we het zwart.

De N_max kan arbitrair gekozen worden maar heeft invloed op de false-positives en dus de 'scherpte' van de fractaal. In theorie zou er geen limiet mogen zijn maar in de praktijk limiteert de resolutie van het beeld ons vermogen om twee punten te onderscheiden binnen dezelfde pixel. Een hoge limiet verlengt dus onnodig de rekentijd. Echter een te lage limiet zorgt ervoor dat punten die nabij de set liggen niet tijdig aan de modulus grens geraken en hierdoor foutief zwart worden gekleurd, met een blob tot gevolg. Punten die dichter bij de set liggen bewegen trager naar oneindig dan punten die er ver van liggen. Het is dan ook nodig om bij dieper zoomen steeds deze limiet te verhogen. We implementeren deze limiet lineair t.o.v. van de scaling van het beeld. De scaling is het aantal pixels per eenheid. Als het beeld ver is ingezoomd dan is de scaling waarde hoog en is de rekentijd dus ook lang. Deze eigenschap van fractalen heeft dus ook verband met de nodige rekentijd voor een gegeven beeld.

3.2 Dynamic parallellism uitbreiding

Dynamic parallellism is een ideale toepassing voor berekeningen zoals deze. Beelden van fractalen kunnen bestaan uit grote regio's die monochroom zijn en regio's die heel complexe patronen bevatten. Afhankelijk van de regio is het dus zinvol om elke pixel individueel te berekenen of slechts enkel de rand van een vlak te berekenen en het midden identiek te kleuren. Het voornaamste probleem met dynamic parallellism in CUDA is dat CUDA ons limiteert tot een queue* van 32 kernel calls. Aangezien we werken met beelden die 16384 bij 16384 pixels zijn, zouden we dit slechts in 32 regio's kunnen verdelen. Voor veel beelden is deze granulariteit te groot om enige tijd uit te sparen of de toegevoegde complexiteit te verantwoorden. Deze ene stap zou ook de enige zijn, aangezien een nieuwe kernel call terug de limiet zou overschrijden.
Indien er geen beperking op het aantal Kernels in de queue zou zijn, zouden we dynamic parallellism zo hebben toegepast. We delen het beeld op in een raster van 32 bij 32 secties. Elke sectie delen we

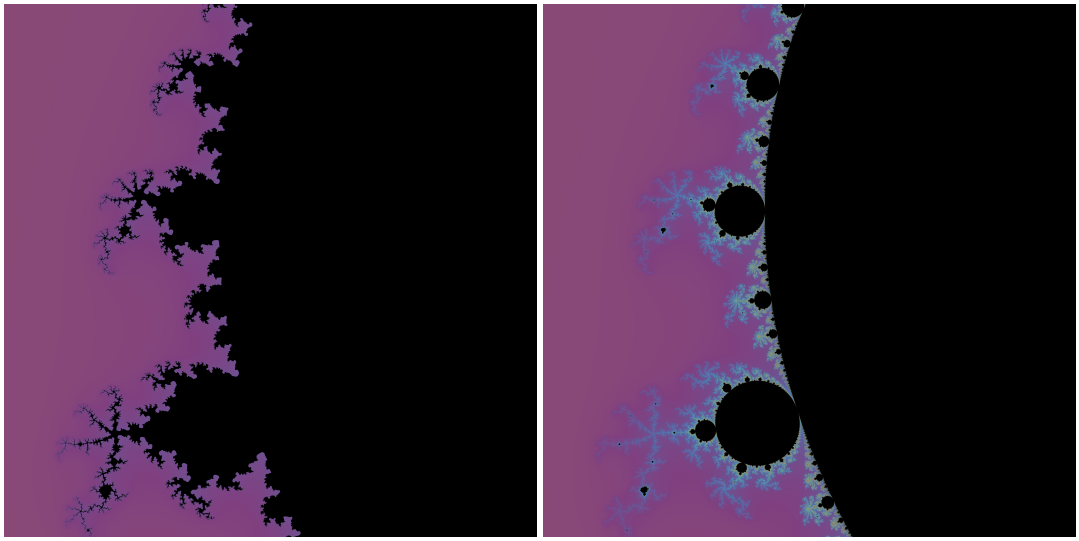

Figure 2: Het verschil tussen een limiet van 32 en 784705 iteraties

terug op in in een raster van 32 bij 32 regio's. Voor elke regio aan de rand van een sectie berekenen we het het kleur van een coördinaat binnen de regio. Als voor alle regio's het kleur hetzelfde is kleuren we heel de sectie in deze kleur. Als echter de kleuren verschillen roepen we de kernel nog eens op voor deze sectie. Deze recursie gaat verder tot één regio overeen met één pixel. Op dat moment wordt voor elke pixel individueel de kleur berekent zonder deze nog op te splitsen in regio's.

4 Resultaten

4.1 Tijdswinst

De nodige tijd om een beeld te berekenen is zoals verwacht afhankelijk van de grootte van het beeld. Voor kleinere afbeeldingen zijn er genoeg cores om alle threads tegelijk te laten lopen. Voor grotere beelden is de looptijd exponentieel aangezien niet alle blocks tegelijk uitgevoerd kunnen worden. Om de vergelijking te kunnen maken hebben we ook voor de CPU een implementatie gebouwd. Deze is analoog aan de implementatie op de GPU maar in plaats van elke pixel een eigen thread toe te kennen, laten we de CPU sequentieel alle pixels berekenen. Zoals verwacht schaalt deze implementatie exponentieel met de breedte van het beeld. Zoals eerder vermeld is de looptijd afhankelijk de schaal van het beeld en de types pixels die zijn afgebeeld. Een ver ingezoomd beeld van pixels in de set van de fractaal zal significant langer duren dan een uitgezoomd beeld waar de fractaal juist niet op staat. Om deze factoren niet de resultaten te laten beinvloeden hebben we voor elke test dezelfde afmetingen en coordinaten gebruikt: de regio in het complex vlak tussen (2,2i) en (-2,-2i). De fractalen zijn zo steeds volledig afgebeeld. Aangezien het wegschrijven naar een file de meest tijdsintensieve stap is hebben we die hier weg gelaten.

Voor kleine beelden is de CPU sneller aangezien deze niet de opstart overhead heeft van de GPU. Voor grotere beelden toont de GPU wel zijn voordeel: de GPU is bijna 2 groot ordes sneller dan de CPU.

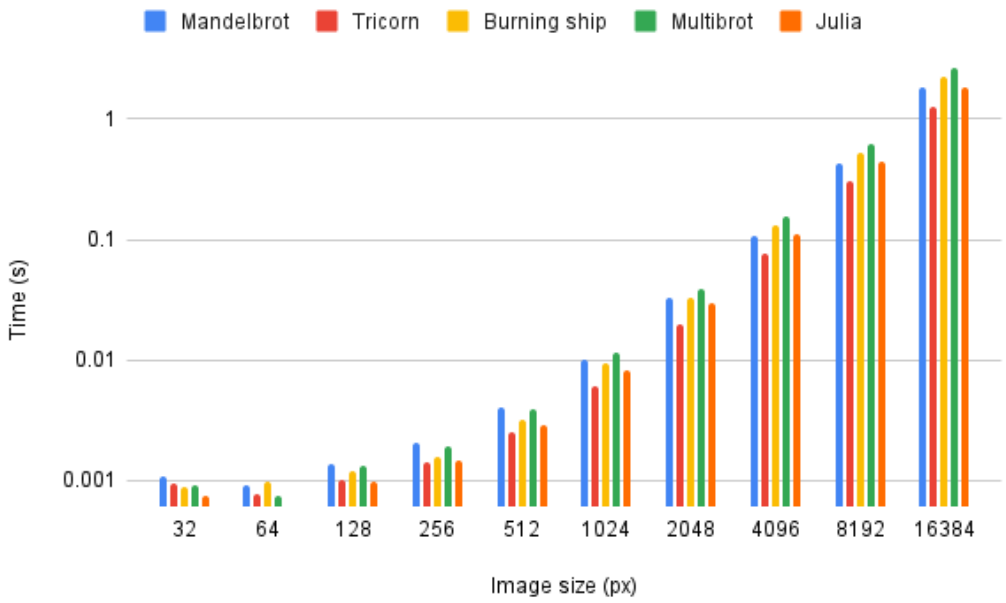

Figure 3: Rekentijd op de GPU in functie van beeld grootte voor verschillende fractalen

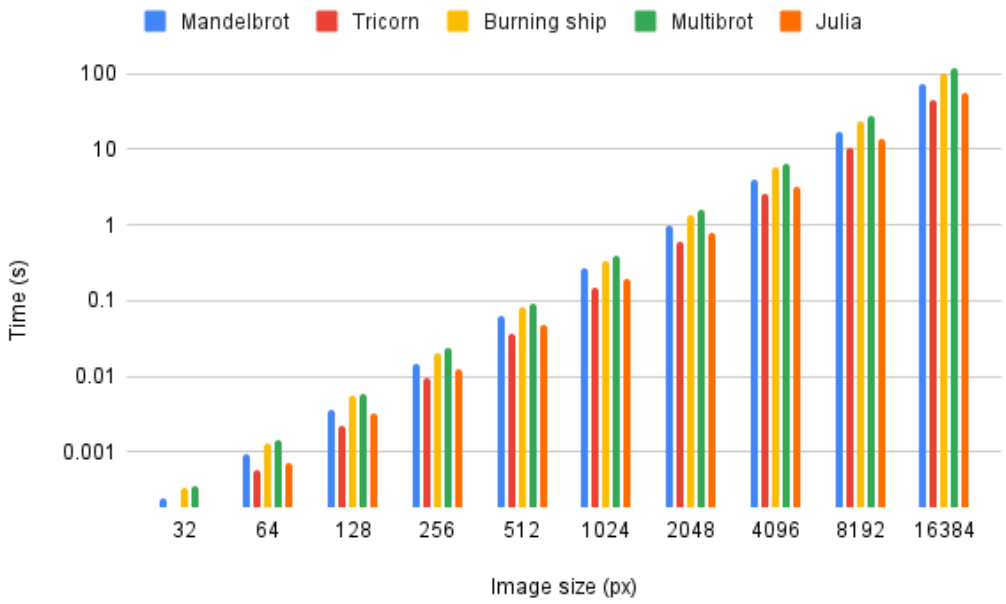

Figure 4: Rekentijd op de CPU in functie van beeld grootte voor verschillende fractalen

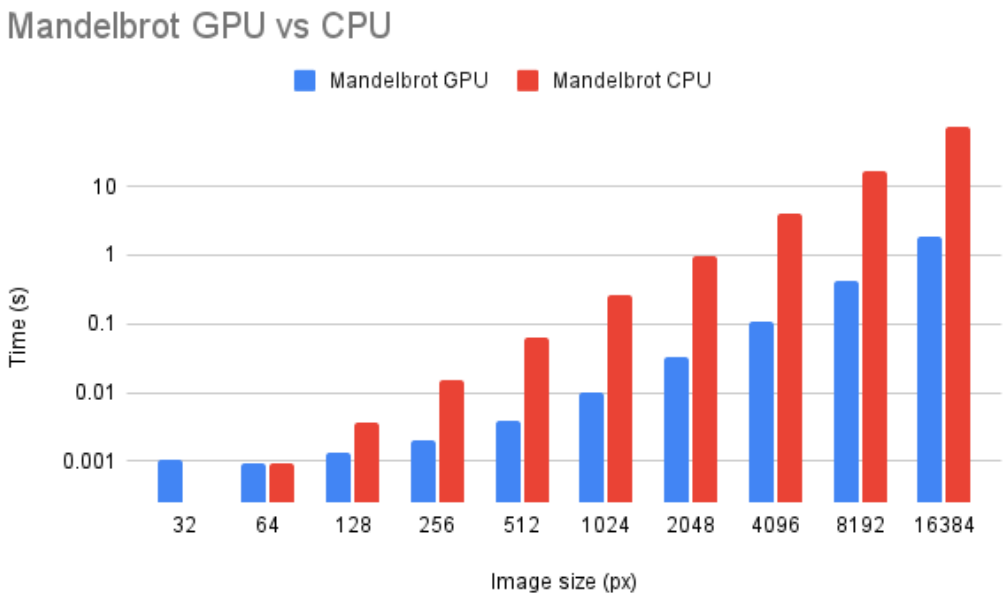

Figure 5: Vergelijking rekentijd GPU en CPU

4.2 Gegenereerde fractalen

In deze sectie tonen we de verschillende fractalen die gegenereerd werden met behulp van de GPU-implementatie.

4.2.1 Mandelbrot-verzameling

De Mandelbrot-verzameling is gedefinieerd door de iteratieve vergelijking:

$$z_{n+1} = z_n^2 + c \tag{1}$$

waarbij $z_0 = 0$ en c een complex getal is dat overeenkomt met een punt in het complexe vlak. Een punt behoort tot de Mandelbrot-verzameling als $|z_n|$ niet divergeert naar oneindig bij herhaalde iteratie.

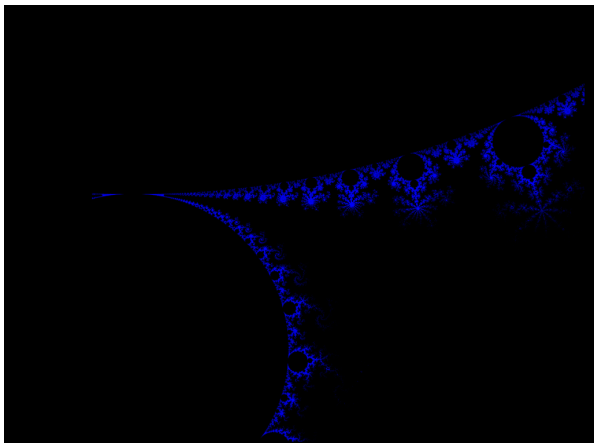

Figure 6: Gegenereerde Mandelbrot-fractaal met GPU

4.2.2 Tricorn

De Tricorn-verzameling lijkt sterk op de Mandelbrot-verzameling, maar gebruikt een aangepaste iteratie waarbij de imaginaire component wordt geconjugeerd:

$$z_{n+1} = \overline{z_n}^2 + c \tag{2}$$

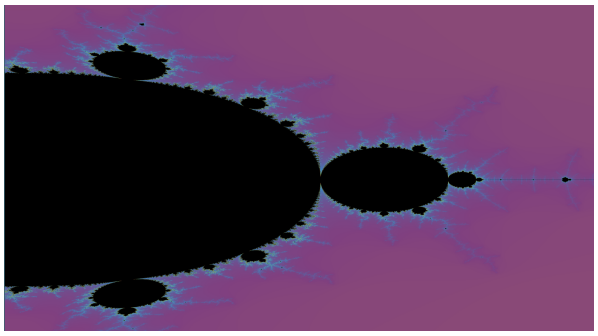

Figure 7: Gegenereerde Tricorn-fractaal met GPU

4.2.3 Burning Ship

De Burning Ship-fractaal gebruikt dezelfde vergelijking als Mandelbrot, maar past eerst een absolute waarde toe op de reële en imaginaire delen van z_n:

$$z_{n+1} = (|\mathrm{Re}(z_n)| + i|\mathrm{Im}(z_n)|)^2 + c \tag{3}$$

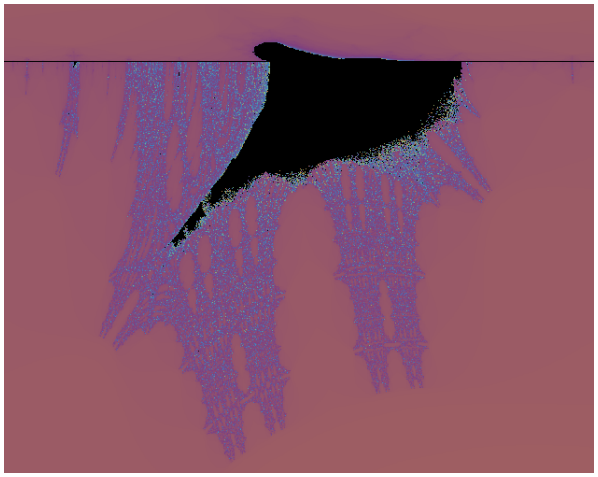

Figure 8: Gegenereerde Burning Ship-fractaal met GPU

4.2.4 Multibrot

De Multibrot-verzameling is een algemene vorm van de Mandelbrot-verzameling, waarbij een hogere macht dan twee wordt gebruikt in de iteratie:

$$z_{n+1} = z_n^d + c \tag{4}$$

In ons geval is $d = 3$, wat een complexer en symmetrischer patroon oplevert.

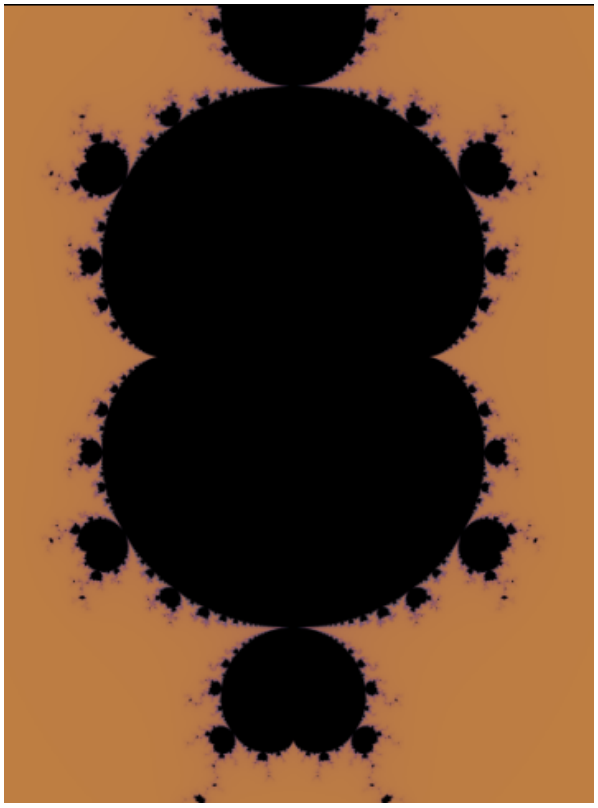

Figure 9: Gegenereerde Multibrot-fractaal ($d = 3$) met GPU

4.2.5 Julia-verzameling

De Julia-verzameling gebruikt dezelfde iteratie als Mandelbrot, maar met een vaste c en een variabele z_0:

$$z_{n+1} = z_n^2 + c \tag{5}$$

Hierbij wordt z_0 gekozen per pixel en blijft c vast, bijvoorbeeld $c = -0.7 + 0.27015i$.

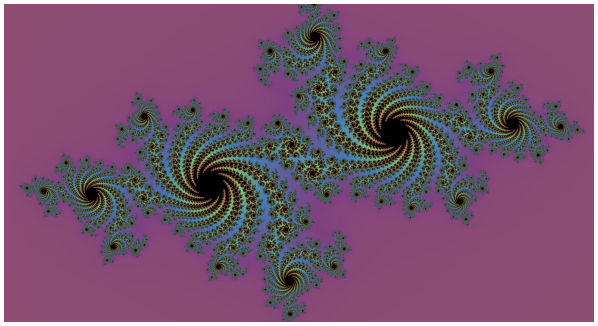

Figure 10: Gegenereerde Julia-fractaal met GPU

5 Mogelijke verbeteringen

5.1 Implementatie van striding

We zouden onze implementatie nog verder kunnen versnellen door striding te gebruiken. Op de GPU stijgt vanaf 512 bij 512 pixels de berekening van de beelden quasi lineair met de oppervlakte, wat er op wijst dat er meer threads zijn dan cores en dat dus warps sequentieel runnen. Aangezien ook elke thread een sterk uiteenlopende rekentijd kan hebben door de chaotische aard van een fractaal, is het zinvol het aantal threads te beperken tot het aantal cores. Elke thread berekent dan d.m.v. striding een collectie van pixels in het beeld. Doordat elke thread kan doorwerken tot al zijn toegekende pixels zijn uitgerekend, treed er geen 'thread divergence' op. De chaos in fractalen werkt hier dan ook in ons voordeel want gemiddeld zullen de tijdsintensieve pixels gelijkmatig verdeeld zijn over alle threads waardoor de totale looptijd niet sterk zal verschillen met de gemiddelde looptijd van alle threads. Hiermee beperken we overhead door thread divergence te elimineren en het wisselen van theads in de cores.

5.2 Memory

In onze huidige implementatie gebruiken we uitsluitend global memory om de pixeldata op te slaan. Een mogelijke optimalisatie is het gebruik van shared memory, dat sneller toegankelijk is binnen één block.
Een specifiek voorbeeld is onze `colorMap`-functie, die voor elke pixel op basis van het aantal iteraties een kleur berekent. In plaats van deze berekening telkens opnieuw uit te voeren, zouden we een kleurentabel (lookup table) kunnen opstellen en deze in shared memory plaatsen.
Op die manier kunnen alle threads binnen een block snel een kleur ophalen op basis van hun `teller`-waarde. Dit zou de rekentijd van de kernel kunnen verminderen, vooral bij grote beelden en veel herhalende waarden.

6 Besluit

De CUDA-gebaseerde fractaalgeneratie leverde een snelheidswinst van factor 80 op dankzij parallelisme. Verdere optimalisatie via dynamic parallelism bleek beperkt door de 32-kernellimiet van CUDA. Striding werd als alternatieve oplossing conceptueel uitgewerkt, maar niet geïmplementeerd.

7 Appendix - code

```
#define STB_IMAGE_WRITE_IMPLEMENTATION
#include "stb_image_write.h"
#include <chrono>    // Timer
#include <cstdint>   // Data types
#include <iostream> // File operations
#include <stdio.h>

#define C 3UL       // Colors
#define SIDELENGTH 4
#define X 2
#define Y 2

// 0.01
// 0.65
//-0.35

void save_image_as_png(uint8_t *image_array, int size) {
 printf("Saving PNG with width: %lu, height: %lu, channels: %lu\n", size, size, C);

 int success = stbi_write_png("output_image.png", size, size, C, image_array, size
 ↪  * C);

 if (success) {
     printf("PNG saved successfully!\n");
 } else {
     printf("ERROR: Failed to save PNG.\n");
 }
}


__device__ uint8_t colorMap(int value, int channel) {
    uint8_t r,g,b;
    value = (value+50)%256;
    r = (value%256 > 127? 64+value%128 : 192-value%128);
    value = value + 64;
    g = (value%256 > 127? 64+value%128 : 192-value%128);
    value = value + 64;
    b = (value%256 > 127? 64+value%128 : 192-value%128);
    if(channel == 0)
        return r;
    if(channel == 1)
        return g;
    else
        return b;
}


__device__ int mandelbrot(double x, double y, double scale, int max_iterations) {
    double c_im = y, c_re = x, z_im = 0, z_re = 0;
    int i = 0;
    while (z_re * z_re + z_im * z_im < 4 && i < max_iterations) {
        double temp = z_re * z_re - z_im * z_im + c_re;
        z_im = 2 * z_im * z_re + c_im;
        z_re = temp;
        i++;
```

```
    }
    return i;
}

__device__ int tricorn(double x, double y, double scale, int max_iterations) {
    double c_im = y, c_re = x, z_im = 0, z_re = 0;
    int i = 0;
    while (z_re * z_re + z_im * z_im < 4 && i < max_iterations) {
        double temp = z_re * z_re - z_im * z_im + c_re;
        z_im = -2 * z_im * z_re + c_im;
        z_re = temp;
        i++;
    }
    return i;
}

__device__ int burning_ship(double x, double y, double scale, int max_iterations) {
    double c_im = y, c_re = x, z_im = 0, z_re = 0;
    int i = 0;
    while (z_re * z_re + z_im * z_im < 4 && i < max_iterations) {
        double abs_re = fabs(z_re);
        double abs_im = fabs(z_im);
        double temp = abs_re * abs_re - abs_im * abs_im + c_re;
        z_im = 2.0 * abs_re * abs_im + c_im;
        z_re = temp;
        i++;
    }
    return i;
}

__device__ int multibrot(double x, double y, double scale, int max_iterations) {
    double c_im = y, c_re = x, z_im = 0, z_re = 0;
    int i = 0;
    while (z_re * z_re + z_im * z_im < 4 && i < max_iterations) {
        double re2 = z_re * z_re;
        double im2 = z_im * z_im;
        double re3 = re2 * z_re - 3 * z_re * im2; // re^3 - 3*re*im^2
        double im3 = 3 * re2 * z_im - im2 * z_im; // 3*re^2*im - im^3
        z_re = re3 + c_re;
        z_im = im3 + c_im;
        i++;
    }
    return i;
}

__device__ int julia(double x, double y, double scale, int max_iterations) {
    // Stel c in op een interessant complex getal, bijv. -0.7 + 0.27015i
    const double c_re = -0.7;
    const double c_im = 0.27015;

    double z_re = x;
    double z_im = y;
    int i = 0;

    while (z_re * z_re + z_im * z_im < 4 && i < max_iterations) {
        double temp = z_re * z_re - z_im * z_im + c_re;
```

```
        z_im = 2.0 * z_re * z_im + c_im;
        z_re = temp;
        i++;
    }
    return i;
}



__global__ void fractalKernel(uint8_t *fractal, double x, double y, double scale,
↪   int size, int fractalType) {
    int max_iterations = 200 + 50 + ((long)scale>>6);

    if (max_iterations < 64) max_iterations = 64;

    double c_im = y - (threadIdx.y + blockDim.y * blockIdx.y)/scale;
    double c_re = x - (threadIdx.x + blockDim.x * blockIdx.x)/scale;

    int teller = 0;
    switch (fractalType) {
        case 1:
            teller = mandelbrot(c_re, c_im, scale, max_iterations);
            break;
        case 2:
            teller = tricorn(c_re, c_im, scale, max_iterations);
            break;
        case 3:
            teller = burning_ship(c_re, c_im, scale, max_iterations);
            break;
        case 4:
            teller = multibrot(c_re, c_im, scale, max_iterations);
            break;

        case 5:
            teller = julia(c_re, c_im, scale, max_iterations);
            break;
        default:
            return;
    }

    int index = (threadIdx.x + blockDim.x * blockIdx.x + (threadIdx.y + blockDim.y *
    ↪   blockIdx.y)*size)*3;

    if(teller == max_iterations){
        fractal[ index + 0] = 0;
        fractal[ index + 1] = 0;
        fractal[ index + 2] = 0;
    } else {
        fractal[ index + 0] =  colorMap(teller,0);
        fractal[ index + 1] = colorMap(teller,1);
        fractal[ index + 2] = colorMap(teller,2);
    }

}
```

```
int main(void) {
        FILE *file = file = fopen("projectgpu.csv", "w");
    if (!file) {
        perror("File opening failed");
        exit(EXIT_FAILURE);
    } else {
        fprintf(stderr, "Writing output to file %s\n", "output.txt");
    }

    for(int k = 1; k < 6; k++){
        printf("\nFractal %i\n", k);
        for(int size = 32; size < 20000; size=size*2){
            printf("%i ", size);
            for(int i = 0; i < 10; i++){
                auto start = std::chrono::steady_clock::now();

                uint8_t *fractal_array = (uint8_t *)malloc(size * size * C *
                ↪  sizeof(uint8_t));

                uint8_t *fractal_array_gpu;
                dim3 dimBlock(32,32);
                dim3 dimGrid(size/32,size/32);

                cudaMalloc((void **)&fractal_array_gpu, size * size * C *
                ↪  sizeof(uint8_t));
                cudaMemcpy(fractal_array_gpu, fractal_array,
                    size * size * C * sizeof(uint8_t), cudaMemcpyHostToDevice);

                double scale = size/SIDELENGTH; // Pixels per unit

                int fractalType = k; // 1: Mandelbrot, 2: tricorn, 3: burning ship,
                ↪  4: multibrot, 5: julia
                fractalKernel<<<dimGrid, dimBlock>>>(fractal_array_gpu, X, Y, scale,
                ↪  size, fractalType);
                cudaDeviceSynchronize();

                cudaMemcpy(fractal_array, fractal_array_gpu,
                    size * size * C * sizeof(uint8_t), cudaMemcpyDeviceToHost);
                cudaFree(fractal_array_gpu);

                long duration =
                ↪  std::chrono::duration_cast<std::chrono::microseconds>(
                    std::chrono::steady_clock::now() - start)
                    .count();

                fprintf(file, "%lu ", duration);
                //save_image_as_png(fractal_array, size);
                free(fractal_array);
            }
            fprintf(file, "\n");
        }
        fprintf(file, "\n");
    }
    return 0;
}
```